```
public static void main (String args[])
{
Student s1 = new student (111, "Karan");
Student s2 = new student (222, "Aryan");
// we can change the college of all objects by
      the single line of code
// student.college = "BBDIT";
   s1.display();
   s2.display();
  }
}
```

Ex:-
```
class A
{
static int x=10;
}
class B
{
public static void main (string args[])
{
system.out.println (A.x);
}
}
```

Abstract classes and Methods.

Data abstraction is the process of hiding certain details and showing only essential information to the user.

> Abstraction can be achieved with either abstract classes or interfaces.

Scanned with OKEN Scanner

> Abstract key is used for classes and methods

Abstract class: It is a restricted class that cannot be used to create objects (to access it, it must be inherited from another class).

> Abstract Method: can only be used in an abstract class, and it does not have a body. The body is provided by the subclass (inherited from).

> Abstract classes are partially implemented class.

Program:

```
abstract class Animal      // Abstract Method (does not have a body)
{
public abstract void animalsound();
 public void sleep()       //Regular method.
   {
    System.out.println("zzz");
   }
}
class Pig extends Animal   //subclass (inherit from Animal)
{
 public void animalsound()  // The body of animalsound() is provided here
   {
    System.out.println("The Pig says: weeweee");
   }
}
class Main
{
 public static void main(String[] args)
   {
```

Scanned with OKEN Scanner

```
Pig myPig = new Pig();
myPig.animalSound();
myPig.sleep();
}
}
```

Y & when to use abstract classes & methods?

⇒ Inter » To achieve security - hide certain details and only show the important details of an object

Note: Abstraction can also be achieved with interfaces.

Animal Method;

> If any method starts with abstract keywords are called abtract Methot are not always having definision and endended with ';'

⇒ Interfaces :-

> To access the interface ∝.

> In Interfaces all methods with only declaration but not having the definitions.

> Interfaces define using interface keywords.

> we cannot create objects for the interfaces.

> To access the interface methods the interface must be implenked by another class with the implements keyword.

Program:

```
interface Animal
{
public void animal sound();
public void sleep();
}
```

Scanned with OKEN Scanner

```
class pig implements Animal
{
  public void animalsound()
  {
  system.out.println("The sound is wee wee");
  }
  public void sleep()
  {
  system.out.println("The sound is : zzz");
  }
}
Class main
{
public static void main(string args[])
 {
pig mypig = new pig();
mypig.animal sound();
 mypig.sleep();
 }
}
```

Differences b/w ~~Abstrac~~ Abstract & Interface classes.

AC: It is partially completed class.

IC: It is fully completed class.

> ~~Acct~~. AC it uses extends keyword(reserve)

> It uses implement (provide) body.

> We can have variables

> Variables are public, static, final by default.

Scanned with OKEN Scanner

> It has constructors
> A class can extend only one abstract class.

> cannot have constructors.
> A class can implement multiple interfaces

> Java does not support multiple inheritance however it can be achieved with interface because the class can implement multiple interfaces.
> To implement multiple interfaces support separate them with (,) comma.

multiple inheritance

Program:

```
interface Animal1
  {
  public void animalsound();
  }
interface Animal2
  {
  public void sleep();
  }
class Pig implements Animal1, Animal2
  {
  public void animalsound()
  {
  system.out.println("The sound is: wee wee");
  }
public void sleep()
  {
system.out.println("The sound is: wee");
}}
```

Scanned with OKEN Scanner

```
class Main3 {
Public static void main(String args[])
{
Pig m = new Pig();
M.animalsound();
m.sleep();
}
}.
```

Command line Arguments:

command line Arguments are nothing but passing the values to java program from command line. They are stored in strings argues parameter of main method.

> were string[] → is array of strings
arg → is array name
arg[0] → first parameter recieved from command prompt.
arg[1] → 2nd parameter recieved from command prompt
arg[2] → 3rd paranet recived from command prompt.

Ex: program:

```
class commandline
{
public static void main(String args[])
{
System.out.println("First argument is:"+arg[0]);
System.out.println("2nd argument is:"+arg[1]);
}
}
```

Scanned with OKEN Scanner

Program 2:

```
import java.util.scanner;
class commandline
{
public static void main(string args[])
{
System.out.println("First argument i's:"+ arg[0]);
System.out.println("and argument i's:"+ arg[1]);
int a = Integer.parseInt(arg[0]);
int b = Integer.parseInt(arg[1]);
  int res = a+b;
System.out.println("sum i's:"+res);
}
}
```

javac class.java

java class args[0] args[1]

Scanned with OKEN Scanner

Interfaces:-

> Interface are the extension
> An Interface is a pure abstract class which contains only abstract methods and final variables
* To the Interfaces object cannot be initialized becau
> TO make the Interface available outside the packag anywhere then the access specifier of interfaces should be public
> The Interface and its methods are by default abstra therefore, they should not be declared with keyword abstract.
> All variables defined in the interface are by default public, static & final variables therefore, they can be accessed using the interface name.
> Interfaces are used for implementing into sub-cla -es and it is the responsibility of subclass that it should override all abstract methods of interfaces otherwise the subclass also becomes an abstract class. object can be instantiated to this subclass to make use of them.

>Multiple Inheritance throw interfaces.

In Java.
Multiple Inheritance cannot be implemented classes where as it can be achieved by using interfaces.

>A class can implement
which is said to be multiple inheritance.

>Extending super class and implementing interface.
It is possible that a circular can extendent super class and implement interfaces but super class should extend first and then interface.

Scanned with OKEN Scanner

```
// Extending class and implementing interface
interface inter1 {
    void sum(int x, int y);
}
class math1 {
    public void sub(int x, int y)
    {
        System.out.print("\n sub = " + (x - y));
    }
}
class myclass extends math1 implements inter1{
    public void sum(int x, int y) {
        system.out.printf("\n sum = " + (x + y));
    }
}
class in3 {
    public static void main(String[] args){
        myclass obj = new myclass();
        obj.sum(10, 20);
        obj.sub(20, 10);
    }
}
```

** Extending interfaces an
> similar to classes and interface can extend another interface.
> The inherited interface contains all abstract methods of its super interface
> Any class that implements inherited interface should implement all abstract methods of sub and super interface.

Scanned with OKEN Scanner

```
// extending interface
interface inter1 {
    void sum(int x, int y);
}
interface inter2 extends inter1 {
    void sub(int x, int y);
}
class myclass implements inter2 {
    public void sum(int x, int y) {
        System.out.println("\n sum = "+(x+y));
    }
    public void sub(int x, int y) {
        System.out.println("\n sub = "+(x-y));
    }
}
class i\A {
    public static void main(String[] args) {
        myclass.obj = new myclass();
        obj.sum(10, 20);
        obj.sub(5, 2);
    }
}
```

> variables and Interfaces.

> variables created interfaces called by default public, static and final. Therefore these variable should be initialized and cannot be modified anywhere

> These variables can be accessed without using the object, but can be accessed using the interface name or implementing class name.

Scanned with OKEN Scanner

// Interface variables are public static & final.

```
interface inter1{
    int A = 100;
}
class myclass implements inter1{
  public void show(){
    System.out.print("\n value of A = "+A);
// A = 200; // error can't modify final variable value
  }
}
Class ind5 {
  public static void main(String[] args){
    myclass obj = new myclass();
    obj.show();
    System.out.print("\n inter1.A = " + inter1.A);
    System.out.print("\n myclass.A = "+myclass.A);
  }
}
```

> Nested Interfaces or Member Interfaces.

>An interface can be declared as a member of a class or other interface is known as nested interface.

> The nested interface should be accessed out-side the class or interface using the outer class name or interface's name.

//Nested interface or Member interfaces

```
class A {
    public interface inter1 {
        public void sum(int x, int y);
```

Scanned with OKEN Scanner

```
    }
}
class B implements A.inter1{
    public void sum(int x, int y){
    System.out.println("In sum="+(x+y));
    }
}
class Int1{
    public static void main(String[] args){
        B.obj=new B();
        obj.sum(5,3);
    }
}
```

⇒ Packages:

> Package is a container of classes,

> packages eliminates names space collision.

> The classes in the packages should be public otherwise, the classes cannot be accesed out side the package.

> The package statement should be first statement in the java program.

> packages are created using package keyword

Program:

```
package pack1;
public class math1
{
public void sum(int x, int y)
{
System.out.print("In sum="+(x+y));
```

Scanned with OKEN Scanner

```
{
public void square (int x)
{
system.out.print("in square =" + (x*x));
}
}
```

→ Importing packages / Accessing packges:

> The classes present in packages can be used in the other programs by importing the classes of packages in other java program.

syntax: `import packagename.classname;`

(or)

`import packagename.*;`

Ex:-

```
import pack1.math1;
class m1
{
public static void main(string args[])
{
pack1.math1 obj = new pack1.math1();
obj.sum(5,60);
obj.square(5);
}
}
```

Scanned with OKEN Scanner

⇒ Access protection:

There are 4 types of access specifiers in java. those are public, private, protected and default.

private:

private members cannot be accessed in sub class of same package default.

protected and public can be accessed.

> private members cannot be accessed in non-sub class of same packages where as default, protected and public can be accessed.

> private, default and protected cannot be accessed in non-sub classes of different package where as protected & public can be accesed.

> The visibility of those access specifiers with different classes in package are shown in the table.

	private	Default	Protected	public
Same class	Yes	Yes	Yes	Yes
same package (non-sub class)	No	Yes	Yes	Yes
Same Package Sub class	No	Yes	Yes	Yes
Different package non-sub class	No	No	No	Yes
Different package sub class	No	No	Yes	Yes

Scanned with OKEN Scanner

Same package and non sub class:

```
> package samplepack;
  public class sample
  {
    private int x=10;
            y= 20; //default access
    protected int z=30;
    public int p=40; // only for compile
  }
```

```
> package samplepack;
  public class myclass // non-sub class
  {
    public void show()
    {
    sample obj=new sample();
    system.out.println("In x value = "+obj.x);
    system.out.println("In y value = "+obj.y);
    system.out.println("In z value = "+obj.z);
    system.out.println("In p value = "+obj.p);
    }
  }
```

>javac -d . myclass.java

↑ capital

```
> import samplepack.myclass;
  class A
  {
    public static void main(String args[])
    {
  samplepack.myclass obj = new samplepack.myclass();
      obj.show();
    }
  }
```

Scanned with OKEN Scanner

same package Sub class:

```
package samplepack;
public class sample
{
 private int x=10;
         y=20;
 protected int z=30;
 public int p=40;
}
package samplepack;
public class myclass extends sample
{
  public void show()
  {
  System.out.println("In X value =", +x);
  System.out.println("In Y value =", +y);
  System.out.println("In Z value =", +z);
  System.out.println("In P value =", +p);
  }
}
import samplepack.myclass;
class A
{
 public static void main(String args[])
 {
 samplepack.myclass obj = new samplepack.myclass();
 obj.show();
 }
}
```

Scanned with OKEN Scanner

Different package and non-sub class:-

If the classes are present in the different package and one class is not subclass of other and in this situation, Inside the non-subclass default and protected no.s cannot be accessed, where as public can be accessed.

→ The private no.s however can't be access the outside class.

program:

```
package samplepack;
public class sample
{
 private int x=10;
         y=20;
 protected int z=30;
 public int p=40;
}
package sampack1;
import samplepack.sample;
public class my.class1
{
 public void show()
 {
 Samplepack.sample obj=new.
                    samplepack.sample();
 System.out.println("In Value of x is-"+obj.x);
 System.out.println("In value of y is="+obj.y);
 System.out.println("In value of z is="+obj.z);
 System.out.println("In value of p is="+obj.p);
}
```

Scanned with OKEN Scanner

```
import samplepack.myclass1;
public class B
{
public static void main (string args[])
{
samplepack1 myclass1 obj = new samplepack.
obj.show();                    myclass();
}
}
```

Different packages and subclass

If the super and subclasses are present in different package, in this situation, inside the subclass default cannot be accessed where as protected and public can be accessed

> private of super class cannot be accessed outside.

Program:

```
package samplepack;
public class sample
{
private int x=10;
        y=20;
protected int z=30;
public int p=40;
}
```

Scanned with OKEN Scanner

Exceptions:

> An Exception in java is one of the java
> Exception is an error (or) unexpected event that occurs during the execution of a program.
> And interrupts the normal flow of the program.
> Exception: Handling of Exceptions is known as Exceptional Handling.
>> Exception handling can be achieved in java using the keywood try, catch, throw, throws, finally.

Syntax:

```
try
{
  // error generated code
}
catch(exceptiontype1 ex1)
{
  // code to handle exception type1
}
catch(exception type2 ex2)
{
  // code to handle exception type2.
}
finally
{
  // final code to execute
}
```

Scanned with OKEN Scanner

1) try block:

The tryblock is used to scan the code for exception during the execution of the program.

> If the exception is occurs, the program flow is interrupted and control goes to catch block.

> A tryblock used for test the code for exception

> A try can have any no.of catch block, and catch blocks should immedietly follows try block

> Each catch block can handle one type of exception, that is reason in tryblock.

2) catch block:-

The exception raised in try block is caught by the catch block, and handle it. so, that the program does not terminate abnormally

3) finally Block: (different from FINAL keyword):

> The finally block executes, if an exception is raised or not raised in try block.

> finally block should be the last block of the try

> A try can contain, any no.of catchy blocks should contain only one finally block.

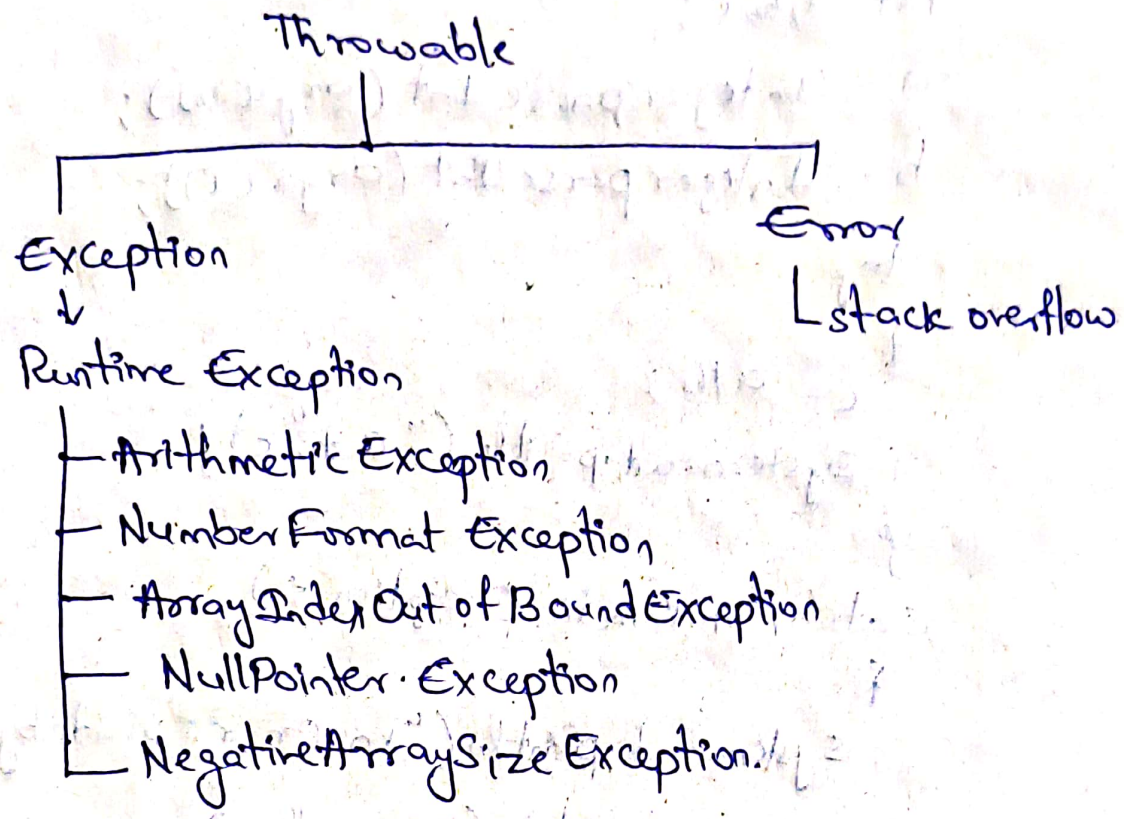
Therefore the try should contain i.e catch or finally or throw

> Exceptions are predefined classes in java which are present in java.lang package

> The super most block for exceptions class is Throwable

Scanned with OKEN Scanner

> This class is inherited into exception and error classes
> The exception class is further inherited into runtime exception.
> The runtime exceptions having sub classes as follows.



Java program to demonstrate Abnormal program termination.

```
class ex1
{
public static void main(String args[])
{
int a,b,c;
a= IntegerparseInt(args[0]);
b = IntegerparseInt(args[1]);
c= a/b;
System.out.println("value of c-"+c);
System.out.println("program terminates successfully");
}
}
```

>javac ex1.java

program to demonstrate Arithmetic exception.

```
// Arithmetic exception
class ex2
{
public static void main(String args[])
{
  int a,b,c;
  a = Integer.parseInt(args[0]);
  b = Integer.parseInt(args[1]);
  try
  {
    c = a/b;
    System.out.println("c = "+c);
  }
  catch(ArithmeticException e)
  {
    System.out.println("In Error: Divide by zero");
    System.out.println("In e: "+e);
  }
  System.out.println("Program continue terminates");
} }
```

> The case of a=10 and b=0, 10 is divisible by 0. is an error. Therefore JVM creates arith. an object of arithmetic exception class

> Finally the JVM throws an arithmetic exception obj which is caught by catch block and the exception object address is assigned to e & catch block is executed. After execution of catch

block the remaining statements till it reaches to main and program terminates normally.

ArrayIndexOutOfBoundException:

Program to demonstrate ArrayIndexOutOfBoundEx

```
// ArrayIndexOutofBoundException.
class ex3
{
public static void main(String args[])
{
 try
 {
 System.out.println("In first argument :"+arg[0]);
 System.out.println("In second argument: "+arg[1]);
 System.out.println("In Third argument :"+arg[2]);
 }
catch(ArrayIndexOutofBoundException e)
 {
 System.out.println("In index out of range :");
 System.out.println("In error message :"+e);
 }
 System.out.println("program terminates");
}
}
```

```
>javac ex3.java
>java ex3 10 20
```

System.out.println("In Third argument :" + arg[2]);

the args does not contain index 2. Therefore JVM identifies error & creates the object of ArrayIndexOutofBoundException & throws it which is

caught by catch block & prints the error message as java.lang.ArrayIndexOutOfBoundException:

* Multiple Catch Blocks:

-> A try can have any number of catch blocks and each catch can handle only one exception

syntax:
```
try
{
// code to scan for exception
}
catch (Exceptiontype1 e1)
{ // code to handle exceptiontype1
catch (Exceptiontype2 e2)
{ // code to handle exceptiontype2
}
finally
{
// final code to execute
}
```

Program:
```
class mcatch
{
public static void main (String args[])
{
int a=0, b=0, c=0;
```

```
try
{
a = Integer.parseInt(argv[0]);
b = Integer.parseInt(argv[1]);
c = a/b;
System.out.println("\n c= "+c);
}
catch(NumberFormatException e1)
{
system.out.print("\n Error 1 : "+e1);
}
catch (Arithmetic Exception e2)
{
system.out.println("\n Error 2: "+e2);
}
catch (IndexOutOfBoundException e3)
{
 system.out.println("\n Error : "+e3);
}
system.out.println("\n Program terminates ---");
}
}
```

Handling Multiple Exceptions.

> It is possible to handle all exceptions by single catch block. A catch block with Exception type can handle all runtime exceptions because runtime exceptions are sub classes of exceptions type

Syntax:

```
try
{
  // code to handle Exceptio
  }
catch (Exception e)
{
 // handle all runtime exceptions
}
```

Program:

```
class mainmcatch1
{
public static void main(String argv[])
{
int a=0, b=0, c=0;
try
{
a = Integer.parseInt(argv[0]);
b = Integer.parseInt(argv[1]);
c = a/b;
System.out.println("In c=" +c);
}
catch(Exception e)
{
System.out.println("erro:"+e);
}
System.out.println("In Program terminates");
}
}
```

Scanned with OKEN Scanner

Note:

```
try
{
  ≡
}
catch(NumberFormatException e1)
{
  ≡
}
catch(ArithmeticException e2)
{
  ≡
}
catch(Exception e3)
{
  ≡
}
```

When a catch block with exception type is used with other catch blocks then catch block with exception type should be last catch block of try.

> In the above example catch block with Exception type handle all exceptions other than NumberFormatException and ArithmeticException.

→ But the following is not correct.

Syntax:

```
try
{
  // code to handle exception
}
catch(Exception e1)
{
```

Scanned with OKEN Scanner

```
}
catch (ArithmeticException e2)
{
//
}
```

Nested try:

> A try can be written in another try is known as nested try.

> Exceptions raised in outer block can be handled by outer try catch block. where as exception raised in inner try can be handled by inner try catch blocks. otherwise the exceptions are reached to outer try catch block and is handled.

Program:

```
class nestty
{
public static void main (String argv[])
{
int a=0, b=0, c=0;
try
{
a= Integer.parseInt(argv[0]);
    try
    {
```

Scanned with OKEN Scanner

```
        b=Integer.parseInt(argv[1]);
        c=a/b;
        System.out.print("\nc="+c);
      }
      catch(ArithmeticException e1)
      {
      System.out.println("\nError1:"+e1);
      }
    }
    catch(NumberFormatException e2)
  {System.out.println("\nError2:"+e2);
    }
    Catch(ArrayIndexOutOfBoundsException e3)
  {
  System.out.println("\n Error3:"+e3);
  }
  System.out.println("\n program terminate--");
  }
}
```

Throwing Exception Manually:

> In the previous programs exceptions are thrown by JVM internally based on the type of exception raised. Java also allows to through the exceptions explicitly in the program. using throw keyword

Syntax:

Throw ThrowableInstance;

ThrowableInst : It should be an object of the class throwable (or) its sub class

The Throw keyword throws the object exception. object explicitly.

program:

```
class throwi
{
public static void main(String argv[])
{
int a=0, b=0, c=0;
try
{
a= Integer.parseInt(argv[0]);
b= Integer.parseInt(argv[1]);
if(b==0) throw new ArithmeticException
("divide by zero");
c=a/b;
System.out.println("\n c="+c);
}
Catch (ArithmeticException e)
{
System.out.println("\nError:"+e);
}
System.out.println("\n program terminates:");
}
```

> A try should contain either catch or finally block.
> A try can contain any number of catch blocks but only one finally block.
> when an exception raises in try block the concerned try catch block executes. otherwise the catch block does not executes.
> Wheras finally block excecutes, if an exception raises or not in any other
> The statements in finally block executes compalsary.

Types of exceptions:

Exceptions classe are two types.
> checked exceptions and unchecked exceptions

(1) Unchecked Exceptions:
> All java methods by defaults have conditions to throws through unchecked except
> when the java compiler comes across on unchecked exception in a method ther compiler does not check any condition with it whether the method can throw the exception or not. therefore called unchecked Exceptions.

Scanned with OKEN Scanner

> ~~Any rel~~

> The unchecked exceptions need not be included in any method of throws list.

Scanned with OKEN Scanner

Checked Exceptions:

> The Java methods do not have permission to throw the checked exceptions by default.

> When the compiler comes across the checked exception in the method we checks the condition whether the method can throw the exception or not therefore it is called checked exceptions.

> The checked exceptions need to be included in the method of throws list otherwise compiler gives an error.

> The checked exceptions classes are listed in below table.

Scanned with OKEN Scanner

program:

```
class checkexp
{
public static void main(string argv[]) throw
                                classNotFoundException
{
System.out.println("\n Sample program on
  checked exception.");

try
{
throw new classNotFoundException("class
                                not found");
}
catch(classNotFoundException e)
{
System.out.print("\n Error:"+e);
}
System.out.println("\n program terminates.");
}
}.
```

to string() method:

> It converts an object's information into a readable string representation.
> printing meaningful object details.

Scanned with OKEN Scanner

program:

```
class student
{ String name;
  int age;
  Student (String name, int age)
  {
   this.name = name;
   this.age = age;
  }
  @override
   public String toString()
  {
   return "student{name="+name+",
                age="+age+"};
   }
}
class A
 {
 public static void main(String args[])
 {
Student s = new student("Raiya", 18);
System.out.println(s);
}
}
```

User defined Exceptions:

Java allows programers to create a
which are called

> User defined exception class should satisfy
to behave like exception class

Scanned with OKEN Scanner

> User defined Exception class should extends from Exception class.
> User defined Exception class should contain toString() method. to return the error message.

Note :-
> User defined exception classes are checked exceptions therefore they should be listed with throws keyword.
> A program to demonstrate creation of user defined exception.
> We write a program to create user defined exception class to handle an exception while creation of bank account. The minimum deposite to create an account is rupees thousand. 1000/-.